Performing Aggregations in Spark

In this train, we'll focus on configuring cluster settings and parameters to optimize performance and resource utilisation in a cloud environment.

Learning Objective

By the end of this train, you should be able to:

- Configure cluster settings and parameters to optimize performance and resource utilization.

Prerequisites

Before proceeding, ensure you are comfortable with the following skills:

- Loading data into Spark from external sources
- Restructuring data using Spark transformations
- Creating additional fields/columns within a dataset
- Generating projections by filtering and selecting data

```
# To start programming in Spark, define a SparkContext and SparkSession.

from pyspark import SparkContext
from pyspark.sql import SparkSession

sc = SparkContext()
spark = SparkSession(sc)
```

```
# Remember to check that the path to the data given below is valid for your
system.
df = spark.read.csv('data/bitcoin_data/bitcoin_train.csv', header=True)
```

Aggregations

The concept of performing an aggregation shouldn't be unfamiliar to us; we've come across these operations in both SQL and Pandas (Python-based DataFrames). Thankfully, we'll soon see that performing an aggregation using PySpark is very similar to these other frameworks.

Remember that when we perform an aggregation, we collect or *'group'* data based on a specified field, and then calculate an aggregate result using a given function. This means that our aggregation operations are inherently composed of two parts:

- The **Group stage**, where we designate which field(s) we should aggregate over; and
- The **Function application stage**, where we specify an aggregation function to apply to our grouped data.

In PySpark, both of these stages can be applied using distinct methods.

Let's first consider the *grouping stage* of an aggregation. To group data together, we call the `groupBy()` method of a DataFrame. This method accepts one or more columns within our DataFrame and returns a `RelationalGroupedDataset` object which can have aggregations applied to it.

To see this in action, let's group our data according to the year it was captured. We could do this by executing the following statement:

```
df.groupBy(F.year('timestamp'))
```

Interesting. As we mentioned above, the result of the `groupBy()` method isn't a DataFrame as we've previously seen, but is instead a `RelationalGroupedDataset` object with specific properties. In fact, we can't actually visualise this object to get a result from it:

```
# Calling .show() on the GroupedData object will result in an error!
df.groupBy(F.year('timestamp')).show()
```

Output:

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
<ipython-input-33-20e299b16339> in <module>
      1 # Calling .show() on the GroupedData object will result in an error!
----> 2 df.groupBy(F.year('timestamp')).show()

AttributeError: 'GroupedData' object has no attribute 'show'
```

This is where the *aggregation function application* stage comes in. It allows us to transform our grouped data object into a calculated result, producing a DataFrame once more.

There are several aggregation functions that we can choose from. These are all specified within and available from our trusty `functions` module, and include:

- `min()`: Return the smallest value found in the grouped data;
- `max()`: Return the largest value found in the grouped data;
- `sum()`: Return the sum of all values in the grouped data; and
- `mean()`: Return the average of all values in the grouped data.

Using this knowledge, let's complete our example from above by counting the number of records that have been recorded for each year within the dataset:

```
# We can call the .count method on the result of our .groupBy operation.
df.groupBy(F.year('timestamp')).count().show()
```

Output:

```
+---------------+------+
|year(timestamp)| count|
+---------------+------+
|           2018|525600|
|           2015|519128|
|           2013|525600|
|           2014|525600|
|           2019|525600|
|           2020|525601|
|           2012|527040|
|           2016|527040|
|           2011|   968|
|           2017|525600|
+---------------+------+
```

Looks good!

It's important to note that we don't always have to use the `groupBy()` function to calculate an aggregate of a column's data. In this sense, there are many aggregation methods which can be called directly from a DataFrame, including `stat()`, `correlation()`, `covariance()`, `sampleBy()`, `approxQuantile()` and `frequentItems()`.

We also aren't restricted to call a single aggregation function on our grouped data. Instead, we can make use of the powerful `agg()` method, which allows us to create multiple aggregations by supplying a dictionary object of all transformations we wish to perform on our DataFrame. Here the basic structure of the dictionary should be `{field: aggregation_function}`, for example, `{'high': 'sum'}`.

Let's see a practical example of this:

```
# Note that we use the .dropna() method to remove all NaN values
# from the dataset which would otherwise cause all our results
# to become NaN.
df.dropna().groupBy(F.year('timestamp')) \
  .agg({'high': 'max', 'volume_currency': 'sum'}) \
  .show(10, False)
```

Output:

```
+---------------+---------+---------------------+
|year(timestamp)|max(high)|sum(volume_currency) |
+---------------+---------+---------------------+
|2018           |17234.99 |3.0746672203400585E10|
|2015           |502.0    |1.5454445301371l2E9  |
|2013           |1163.0   |1.5511774873354862E9 |
|2014           |995.0    |2.6152928091775055E9 |
|2019           |13880.0  |2.3158536096590027E10|
|2020           |29010.32 |3.3017603788313705E10|
|2012           |16.41    |5764800.229803922    |
|2016           |980.74   |1.1124637970242171E9 |
|2011           |4.58     |425.3203430175781    |
|2017           |19666.0  |2.1775822445412468E10|
+---------------+---------+---------------------+
```

Looking at these results, we can make two observations:

Firstly, *the names of the aggregated columns conform to a specific format*, namely `aggregation_function(column_name)`. For example, using the `max` function on the `'high'` column produces a result column called `'max(high)'`.

Secondly, it becomes apparent that any trends in our results are difficult to spot due to a lack of order. This is where the `orderBy()` method saves the day. Almost identical to its SQL counterpart, in Spark the `orderBy()` method accepts a column name used to

order the aggregation output either in ascending (default behaviour) or descending order.

We can extend our example once again by adding in ordering:

```
# Take note that, because of Spark's naming convention,
# we order by 'year(timestamp)' instead of just 'timestamp'.
df.dropna().groupBy(F.year('timestamp')) \
    .agg({'high': 'max', 'volume_currency': 'sum'}) \
    .orderBy('year(timestamp)').show(10, False)
```

Output:

```
+--------------+---------+--------------------+
|year(timestamp)|max(high)|sum(volume_currency) |
+--------------+---------+--------------------+
|2011           |4.58     |425.3203430175781    |
|2012           |16.41    |5764800.229803922    |
|2013           |1163.0   |1.5511774873354862E9 |
|2014           |995.0    |2.6152928091775055E9 |
|2015           |502.0    |1.5454445301371 12E9  |
|2016           |980.74   |1.1124637970242171E9 |
|2017           |19666.0  |2.1775822445412468E10|
|2018           |17234.99 |3.0746672203400585E10|
|2019           |13880.0  |2.3158536096590027E10|
|2020           |29010.32 |3.3017603788313705E10|
+--------------+---------+--------------------+
```

As can be seen in the code above, Spark's naming convention can become pretty unwieldy quickly. Here if we were to string multiple aggregations together on a single field, it could result in something like `agg_3(agg_2(agg_1(field_name)))`. Only a mother could love that.

Spark helps us prevent such a tragedy by providing the `alias()` method, which allows us to specify what the result of an aggregation should be called. To create aliases, we use a select statement to get the columns that we want to rename, followed by using the `col()` method to retrieve the desired columns, and `alias()` method to rename them. As we've seen by now, Spark gives us multiple ways to accomplish something. You can also use the `withColumnRenamed()` method here instead of `alias()`. In most cases, it will depend on the use case while optimising succinctness, which determines what our approach should be.

Let's put this all together by creating one final aggregation to give us a sense of Bitcoin's behaviour over the last 10 years:

```
# We chain the .alias() method within a select statement in order
# to rename our aggregated results.
df.dropna().groupBy(F.year('timestamp'))\
    .agg({'high': 'max', 'volume_currency': 'sum'})\
    .select(F.col('year(timestamp)').alias('year_agg'),
            F.col('max(high)').alias('max'),
            F.col('sum(volume_currency)').alias('volume_sum'))\
    .orderBy('year(timestamp)').show(10, False)
```

Output:

```
+--------+--------+--------------------+
|year_agg|max     |volume_sum          |
+--------+--------+--------------------+
|2011    |4.58    |425.3203430175781   |
|2012    |16.41   |5764800.229803922   |
|2013    |1163.0  |1.5511774873354862E9|
|2014    |995.0   |2.6152928091775055E9|
|2015    |502.0   |1.545444530137112E9 |
|2016    |980.74  |1.1124637970242171E9|
|2017    |19666.0 |2.1775822445412468E10|
|2018    |17234.99|3.0746672203400585E10|
|2019    |13880.0 |2.3158536096590027E10|
|2020    |29010.32|3.3017603788313705E10|
+--------+--------+--------------------+
```

This is quite informative. At first, we can observe Bitcoin's peak in 2017 where the digital currency maxed out just under 20 000 USD, then its subsequent crash, and finally, its recent climb and resurgence to a new high of just under 30 000 USD in 2020. We can also see a correlation between Bitcoin's trading value and the volume of trades over time. This isn't bad, considering we've used just a few basic Spark commands to analyse our data.

Conclusion

In this session, we explored how to perform powerful aggregation operations in PySpark, a critical step in analyzing and drawing insights from large-scale datasets. We learned how to group data, apply aggregate functions, customize output formatting, and enhance readability through ordering and aliasing. These skills are essential for summarizing trends and uncovering meaningful patterns in big data.

▼Key Takeaways

- **Takeaway 1:** Aggregation Structure, every aggregation in Spark involves a grouping stage and an aggregation function application stage.
- **Takeaway 2:** Using `groupBy()` and `agg()`, the `groupBy()` method creates grouped datasets, while `agg()` allows for applying multiple aggregate functions simultaneously.
- **Takeaway 3:** Improving Readability, use `orderBy()` to sort results and `alias()` to rename output columns for better clarity.
- **Takeaway 4:** Flexibility in Aggregations, PySpark supports a wide range of built-in aggregation functions and offers several approaches to customize and optimize data summaries.